



UrbanPulse - Urban Mobility Analytics System

Samsung Innovation Campus Hackathon

SET -A

A city transport department has collected 3 years of bus ridership data across 50 routes. Your team must analyse peak loads, detect anomalies, and build an interactive zone map to help planners reduce congestion.

Code Mavens

Nikhil Bhosale	-	RA2311031010035
Lavanya Joshi	-	RA2311031010036
Sagnik Sannigrahi	-	RA2311003011709

Table of Contents

Chapter No.	Title	Page No.
1	Introduction	3
2	System Requirements and Architecture	4
3	Project Pipeline	6
4	System Design	8
5	Task 1 – Data Cleaning and ETL	11
6	Task 2 – Route Efficiency Ranking	12
7	Task 3 – Data Visualization	13
8	Task 4 – Interactive Mobility Map	14
9	Results and Outputs	15

CHAPTER 1

INTRODUCTION

Urban transportation systems generate large volumes of operational data every day. However, such data often contains missing values, inconsistent formatting, outliers, and invalid records that reduce the quality of analysis. This project develops a complete Urban Mobility Analytics System capable of cleaning noisy transportation data, performing route efficiency analysis, generating visual dashboards, and displaying passenger density on an interactive city map.

The project demonstrates the complete data analytics workflow beginning from raw data generation to final visualization using Python libraries such as Pandas, NumPy, Matplotlib, and Folium. A custom Merge Sort algorithm is implemented to satisfy the algorithmic requirements of the hackathon.

The system follows a modular architecture where each processing stage performs a specific task before passing the processed data to the next stage. Realistic transportation records are generated synthetically and intentionally corrupted to simulate common real-world data quality issues. The ETL process ensures that missing values, inconsistent entries, and statistical outliers are handled systematically while preserving the integrity of the dataset. After preprocessing, meaningful features such as peak load classification and day of the week are derived to support further analysis. The cleaned data is then grouped by transportation route to compute average passenger counts and average delays. A custom Merge Sort algorithm ranks the routes based on an efficiency score that considers both passenger volume and service delays. Analytical dashboards provide clear visual insights into passenger trends, delay distributions, and zone-wise traffic patterns. An interactive city map further enhances understanding by displaying passenger density and congestion across different zones. The project emphasizes modular design, code reusability, and efficient data processing, making it suitable for extension into real-time transportation monitoring systems. Overall, the developed solution demonstrates the practical application of data cleaning, algorithm design, statistical analysis, and geospatial visualization within a single integrated analytics platform.

CHAPTER 2

SYSTEM REQUIREMENTS AND ARCHITECTURE

The Urban Mobility Analytics System is developed using Python and utilizes several open-source libraries for data processing, visualization, and geospatial analysis. The software requirements for successfully executing the project are as follows:

- **Python 3.10 or Higher:** Serves as the primary programming language for implementing the complete analytics pipeline, including data generation, cleaning, algorithm implementation, visualization, and map creation.
- **Visual Studio Code (VS Code):** Used as the Integrated Development Environment (IDE) for writing, debugging, and executing the Python source code. Its built-in terminal and extension support simplify project development.
- **Pandas:** Provides powerful data structures and functions for data manipulation, preprocessing, cleaning, aggregation, grouping, and exporting the processed dataset.
- **NumPy:** Used for efficient numerical computations, random dataset generation, statistical calculations, array operations, and outlier detection.
- **Matplotlib:** Generates high-quality analytical visualizations such as line charts, bar graphs, histograms, and scatter plots, which are combined into the project dashboard.
- **Folium:** Creates interactive web-based maps that display transportation zones, passenger density, congestion levels, and heatmap visualizations using geographical coordinates.
- **Standard Python Libraries:** Built-in modules such as **math** and **datetime** are utilized for mathematical computations, date manipulation, and other utility operations required throughout the project.

This software stack provides a lightweight, cross-platform development environment and requires no additional database servers, web frameworks, or containerization tools, making the application easy to set up and execute on any system with Python installed.

The complete workflow is summarized below:

1. **Raw Data Generation:** Generate a synthetic dataset containing transportation records such as route IDs, passenger counts, delays, dates, and city zones.
2. **Noise Injection:** Introduce intentional inconsistencies including missing values, dirty strings, whitespace, invalid casing, sentinel values, and statistical outliers to mimic real-world data.

3. Data Cleaning (ETL): Clean and preprocess the dataset by converting values to appropriate data types, handling missing values, removing inconsistencies, detecting outliers, and creating additional derived features.
4. Route Statistics Computation: Group records by route and calculate average passenger count and average delay for each transportation route.
5. Route Efficiency Ranking: Implement a custom Merge Sort algorithm to rank transportation routes according to an efficiency score based on passenger volume and delay.
6. Dashboard Generation: Produce graphical visualizations including passenger trends, zone-wise analysis, delay distribution, and passenger-delay relationships.
7. Interactive Map Generation: Create an interactive Folium map displaying transportation zones, passenger density, and congestion using markers and heatmaps.
8. Output Generation: Export the cleaned dataset, analytical dashboard, and interactive map as project deliverables.

CHAPTER 3

PROJECT PIPELINE

The Urban Mobility Analytics System follows a structured and sequential processing pipeline to transform raw transportation data into meaningful analytical insights. Each stage of the pipeline performs a specific function, beginning with synthetic data generation and progressing through data preprocessing, feature engineering, route analysis, visualization, and final output generation. This modular workflow ensures that the dataset is systematically cleaned, analysed, and visualized while maintaining accuracy, scalability, and ease of maintenance.

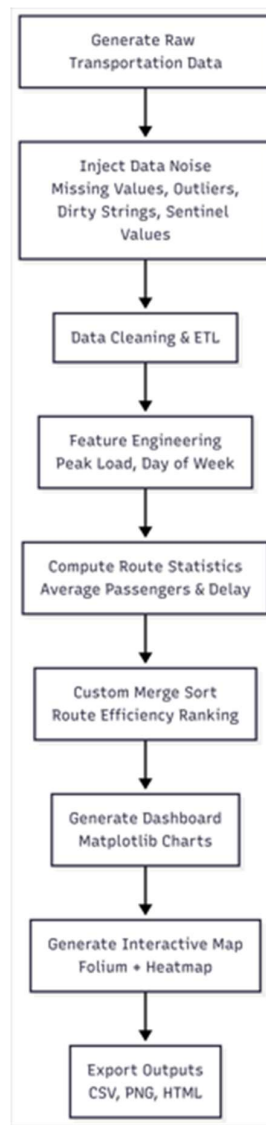


Fig 3.1 : Pipeline Flowchart for UrbanPulse

Step 1: Generate synthetic transportation dataset.

Step 2: Inject realistic data quality issues including missing values, dirty strings, inconsistent casing, whitespace, sentinel values, and outliers.

Step 3: Perform ETL by cleaning every affected column.

Step 4: Create derived attributes including Peak Load and Day of Week.

Step 5: Compute average passenger count and average delay for every route.

Step 6: Rank routes using a custom Merge Sort implementation.

Step 7: Generate statistical dashboard using Matplotlib.

Step 8: Generate interactive city mobility map using Folium.

Step 9: Export cleaned dataset and visualization files.

CHAPTER 4

SYSTEM DESIGN

The System Design chapter provides a graphical representation of the overall functionality and workflow of the Urban Mobility Analytics System. System modelling helps visualize how users interact with the application and how data flows through different processing stages. The diagrams presented in this chapter illustrate the major functional components of the system and the sequence of operations performed during execution.

4.1 Use Case Diagram

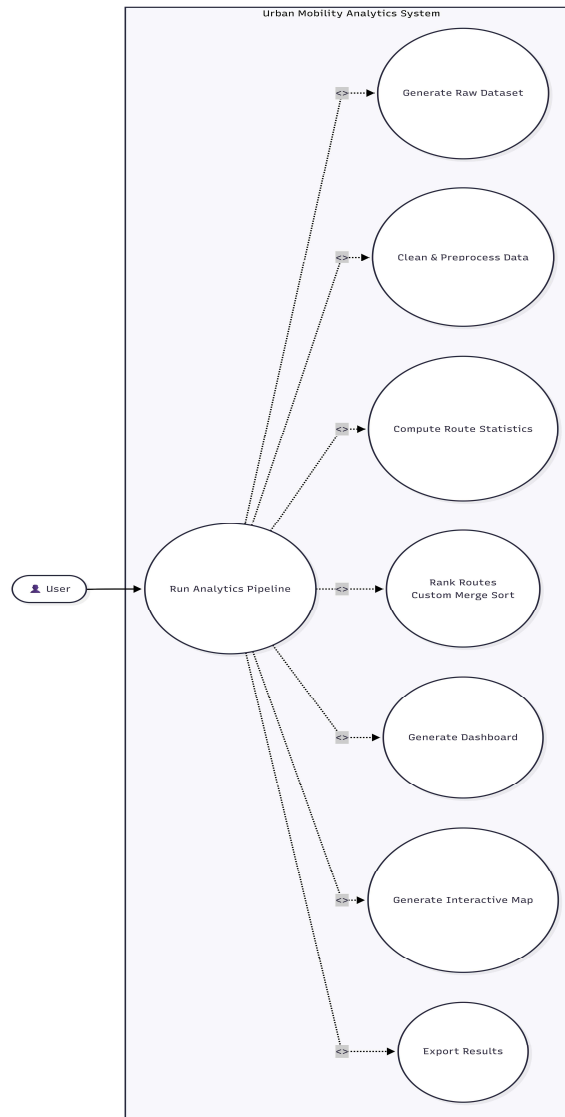


Fig 4.1: Use Case Diagram of UrbanPulse

The Use Case Diagram illustrates the interaction between the User and the Urban Mobility Analytics System, highlighting the major functionalities performed by the application. The user initiates the execution of the system by generating a synthetic transportation dataset, after which the system cleans and preprocesses the data by handling missing values, inconsistencies, outliers, and invalid entries. It then computes route-wise statistics, ranks transportation routes using a custom Merge Sort algorithm based on an efficiency score, generates analytical dashboards using Matplotlib, and creates an interactive mobility map using Folium to visualize passenger density and congestion across different city zones. Finally, the system exports the cleaned dataset, dashboard, and interactive map as output files. The Use Case Diagram provides a high-level overview of the complete functionality of the application and demonstrates how the user interacts with the different modules to perform end-to-end urban mobility data analysis.

4.2 UML Diagram

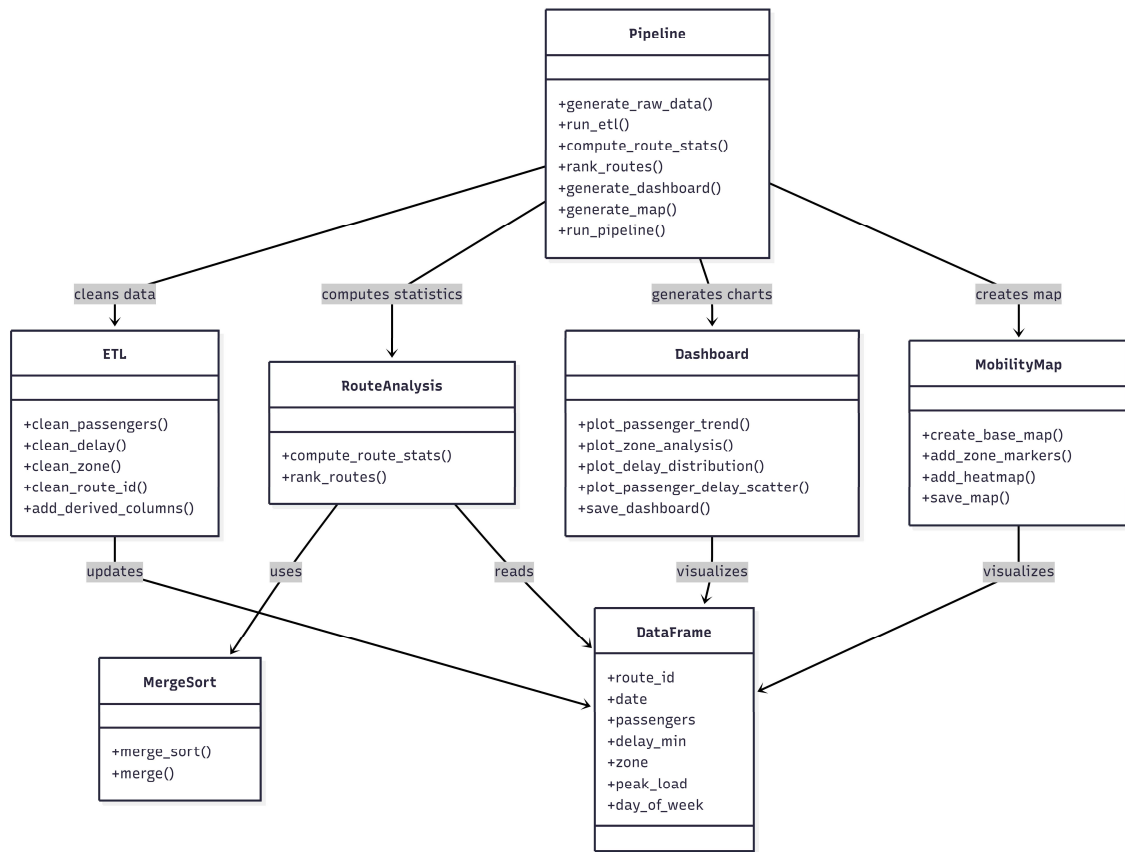


Fig 4.2 : UML Diagram of UrbanPulse

The UML Class Diagram illustrates the structural organization of the Urban Mobility Analytics System by representing its major functional modules and their relationships. The Pipeline class

acts as the central controller, coordinating the execution of all processing stages from raw data generation to output creation. The ETL class is responsible for cleaning and preprocessing the transportation dataset, while the RouteAnalysis class computes route-wise statistics and utilizes the MergeSort class to rank routes according to their efficiency score. The Dashboard class generates analytical visualizations using Matplotlib, whereas the MobilityMap class creates an interactive city map using Folium. The DataFrame class represents the cleaned transportation dataset that is shared among different modules for analysis and visualization. Together, these classes demonstrate the modular architecture of the system and the interactions between its core components.

CHAPTER 5

TASK -1 DATA CLEANING AND ETL

The first stage of the Urban Mobility Analytics System focuses on data cleaning and ETL (Extract, Transform, Load), which is a crucial step in ensuring the reliability and accuracy of all subsequent analysis. The raw transportation dataset is intentionally noisy and contains inconsistencies such as missing values, invalid formats, and statistical anomalies. Therefore, a systematic preprocessing pipeline is applied to transform the raw data into a clean and structured format suitable for analysis.

The following operations are performed during this stage:

- Passenger values, which may contain dirty strings such as “2443 pax” or invalid entries, are converted into proper numeric format using pattern extraction and type coercion techniques.
- Missing values in the passenger column are handled by replacing them with the median value, ensuring that extreme values do not distort the dataset distribution.
- Statistical outliers in passenger data are identified using the Z-score method, which detects values that deviate significantly from the mean of the dataset.
- Identified outliers are not removed but are instead capped at the 99th percentile, preserving data continuity while reducing the impact of extreme spikes.
- Sentinel values in the delay column (such as -999) are treated as invalid and are replaced with null values to ensure consistency in delay analysis.
- Missing delay values are then filled using a route-wise median strategy, which ensures that imputation is context-aware and reflects route-specific behavior rather than global averages.
- Zone names are standardized by converting them into a uniform format (Title Case) and validating them against predefined valid zones to eliminate inconsistencies.
- Route identifiers are cleaned by removing unnecessary leading and trailing whitespace, ensuring uniform identification of routes across the dataset.
- A new derived feature called Peak Load classification is created by categorizing passenger volume into Low, Medium, and High traffic levels based on predefined thresholds.
- The Day of Week feature is extracted from the date column to enable temporal analysis of transportation patterns across weekdays and weekends.

After completing all preprocessing operations, the cleaned dataset is exported as:

`cleaned_ridership.csv`

This cleaned dataset forms the foundation for all further analytical tasks, including route efficiency computation, visualization, and geospatial mapping. It ensures that the downstream modules operate on consistent, reliable, and structured data, significantly improving the accuracy and interpretability of the final results.

CHAPTER 6

TASK - 2: ROUTE EFFICIENCY RANKING (MERGE SORT)

The second stage of the Urban Mobility Analytics System focuses on evaluating and ranking transportation routes based on their operational efficiency. After completing the data cleaning process, the dataset is grouped by Route ID to derive meaningful route-level insights. This aggregation step helps convert individual trip-level records into a structured summary that represents overall route performance.

For each route, two key statistical measures are computed: the average number of passengers and the average delay in minutes. These metrics provide a balanced view of both demand (passenger load) and service quality (delay performance), which are essential for evaluating transportation efficiency.

To satisfy the algorithmic requirement of the hackathon, the project implements a custom Merge Sort algorithm instead of using Python's built-in sorting functions. This ensures a manual and transparent implementation of the divide-and-conquer sorting approach, demonstrating core data structure knowledge and algorithm design skills.

The route efficiency score is then calculated using the following formula:

$$\text{Efficiency Score} = \text{Average Passengers} / (1 + \text{Average Delay})$$

This formula ensures that routes with high passenger demand and low delays are ranked higher, while routes with higher delays are penalized in the scoring system. The addition of 1 in the denominator prevents division errors and stabilizes the score for routes with very low delays.

Once efficiency scores are computed for all routes, the custom Merge Sort algorithm is applied to rank the routes in descending order of efficiency. This sorting process ensures an optimal time complexity of $O(n \log n)$ while maintaining stability in ordering.

Finally, the system generates meaningful insights by displaying:

- The top five most efficient routes, representing the best-performing transportation routes in terms of passenger handling and punctuality.
- The bottom five least efficient routes, which highlight routes that may require operational improvements or further investigation.

This ranking module plays a crucial role in the system by transforming raw aggregated data into actionable insights that can support decision-making in urban transportation planning and optimization.

CHAPTER 7

TASK – 3 VISUALIZATION

The third stage of the Urban Mobility Analytics System focuses on transforming processed numerical data into meaningful visual insights using graphical representations. After completing route-level analysis and efficiency ranking, the cleaned dataset is used to generate multiple visualizations that help in understanding patterns, trends, and distributions in urban transportation behavior.

This stage is implemented using the Matplotlib library, which provides powerful tools for creating static, high-quality plots suitable for analytical dashboards. The objective of this module is to enhance interpretability by converting raw statistical outputs into intuitive visual formats.

Several types of visualizations are generated as part of this task. A passenger trend analysis plot is created to observe how passenger counts vary over time, helping identify seasonal or temporal fluctuations in transportation demand. Additionally, a zone-wise passenger distribution chart is generated to compare traffic intensity across different city zones such as North, South, East, West, and Central.

To understand operational efficiency, a delay distribution plot (histogram) is used to analyze how frequently different delay ranges occur across the dataset. This helps in identifying whether delays are consistently low or if there are frequent high-delay events affecting system performance.

Furthermore, a scatter plot between passengers and delay is created to examine the relationship between demand and service efficiency. This visualization helps determine whether higher passenger loads are associated with increased delays, which is a critical insight for transportation planning.

All individual plots are combined into a single analytical dashboard, ensuring a consolidated view of system performance metrics. This dashboard is then exported as an image file:

mobility_dashboard.png

CHAPTER 8

TASK – 4 INTERACTIVE ZONE MAP

The fourth stage of the Urban Mobility Analytics System focuses on the spatial visualization of transportation data through an interactive geospatial map. This module is implemented using the Folium library, which enables the creation of dynamic, web-based maps that effectively represent geographical patterns and transportation intensity across different city zones.

The primary objective of this stage is to provide a visual understanding of passenger distribution and congestion levels across the city, allowing users to interpret spatial trends that are not easily observable through tabular or numerical data alone.

The system assigns geographical coordinates to predefined city zones such as North, South, East, West, and Central. These coordinates are then used as reference points for plotting data-driven visual elements on the map.

Several visualization features are incorporated into this module. Circle markers are placed for each zone to represent transportation activity, where the size of each marker is dynamically scaled based on the average number of passengers in that zone. This ensures that zones with higher passenger traffic are visually emphasized.

In addition, color encoding is used to represent passenger density levels, where different colors indicate low, medium, and high traffic conditions. This enhances visual interpretability and allows quick identification of congested regions.

Each marker also includes interactive popups that display key statistical information such as average passengers, delays, and zone identifiers. This interactivity allows users to explore detailed insights by simply clicking on a specific region.

Furthermore, a HeatMap layer is added to represent overall congestion intensity across the city. This layer provides a smooth gradient-based visualization that highlights high-density transportation areas, making it easier to identify traffic hotspots.

The final interactive map is exported as an HTML file:

`city_mobility_map.html`

This file can be opened in any web browser and allows users to interact with the visualization without requiring any additional software installation.

Overall, this module enhances the analytical capabilities of the system by introducing a geographical perspective to transportation data, enabling better decision-making in urban mobility planning and congestion analysis.

CHAPTER 9

RESULTS AND OUTPUT

The successful execution of the Urban Mobility Analytics System produces multiple output files that represent the results of each stage of the analytics pipeline. These outputs include the cleaned dataset, analytical dashboard, and interactive mobility map, each serving a distinct purpose in the overall analysis process.

The generated output files are as follows:

- `cleaned_ridership.csv` – Contains the fully cleaned and pre-processed transportation dataset with standardized values, missing data handled, outliers treated, and additional derived attributes such as Peak Load and Day of Week.
- `mobility_dashboard.png` – A comprehensive dashboard consisting of multiple statistical visualizations, including passenger trends, zone-wise passenger distribution, delay analysis, and passenger-delay relationships. These charts provide valuable insights into transportation performance and operational patterns.
- `city_mobility_map.html` – An interactive web-based map generated using the Folium library that displays transportation zones, passenger density, congestion levels, and heatmap visualizations. Users can interact with the map to explore detailed statistics for each zone through informative popups.

The cleaned dataset serves as the foundation for accurate analysis by eliminating inconsistencies and ensuring data integrity. The dashboard enables users to interpret transportation statistics through intuitive graphical representations, while the interactive mobility map provides a geographical perspective of passenger movement and congestion across different city zones. Together, these outputs demonstrate the successful completion of the complete analytics pipeline.